第15章 一人公司实战案例（内容创作/社群运营）

> 💡 **本章目标**：通过2个真实的一人公司案例，展示如何用OpenClaw实现商业闭环，从内容创作到社群运营，从0到1跑通完整业务流程。

🎯 本章内容

- 15.1 案例1：10分钟完成全平台内容发布
- 15.2 案例2：1天冷启动100人付费社群
- 15.3 一人公司的核心方法论
- 15.4 可复制的自动化模板

15.1 案例1：10分钟完成全平台内容发布

15.1.1 案例背景

创作者：某内容创作者（AI编程领域）
痛点：传统内容创作流程耗时3小时
目标：实现从选题到全平台发布的自动化
效果：3小时 → 10分钟，效率提升94%

15.1.2 传统流程 vs 自动化流程

传统流程（3小时）：

```

选题（30分钟）
  ↓
写作（1-2小时）
  ↓
排版+配图（30分钟）
  ↓
各平台手动发布（30分钟）
```

自动化流程（10分钟）：

```

Agent每天9点推送5个选题 → 选一个（1分钟）
  ↓
Agent 5分钟出初稿 → 人工审核修改（5分钟）
  ↓
自动推送飞书 → 格式已排好
  ↓
字流一键发布 → 14个平台（4分钟）
```

15.1.3 核心架构：墨笔Agent

Agent定位：

- 只管内容（图文、短视频、社交媒体）
- 有自己的人格（嗅觉敏锐、表达锋利、有主见）
- 读过所有历史文章（知道写作风格、用词习惯、标题偏好）
- 绑定工具链（飞书API、发布工具API、社交媒体API）

关键配置文件：

1. **SOUL.md** - Agent的角色和边界
2. **写作风格文件** - 3000字的风格规范
3. **记忆系统** - 每天的工作记录

15.1.4 自动选题推送系统

定时任务配置：

```
# 每天早上9点自动执行
0 9 * * * /path/to/openclaw run daily-topic-push
```

选题生成流程：

```
1. 用搜索工具搜索社交媒体过去24小时的AI热点
   ↓
2. 用Brave Search搜索Google最新资讯
   ↓
3. 结合定位（AI编程+出海+创业）筛选5个选题
   ↓
4. 每个选题生成：
   - 备选标题
   - 热度评分
   - 切入角度
   - 核心要点
   ↓
5. 推送到Telegram
```

实际效果：

```
你：[早上9点收到推送]

Agent：今日选题推荐：

1. 【热度⭐⭐⭐⭐⭐】Claude Opus 4.6发布
   标题：Claude一个插件，让全球软件股蒸发2850亿美元
   角度：从技术突破到市场影响
   要点：性能提升、应用场景、行业冲击

2. 【热度⭐⭐⭐⭐】OpenAI Codex独立App上线
   标题：OpenAI放大招：一次跑10个AI Agent帮你写代码
   角度：产品体验+实战测试
   要点：功能亮点、使用体验、对比分析

3. 【热度⭐⭐⭐】AI编程工具安全漏洞
   标题：280+ Leaky Skills：OpenClaw暴露API 密钥
   角度：安全警示+防范措施
   要点：漏洞分析、影响范围、解决方案

4. 【热度⭐⭐⭐】Vibe Coding已死
   标题：Karpathy说未来叫Agentic Engineering
   角度：编程范式变革
   要点：概念解读、实践案例、未来趋势

5. 【热度⭐⭐】OpenClaw 2.6调教实录
   标题：从崩溃4671次到省50% token
   角度：实战经验分享
```

```
    要点：踩坑记录、优化方法、效果对比

回复数字选择，如：写 1

你：写 1

Agent：收到！开始创作《Claude一个插件，让全球软件股蒸发2850亿美元》
```

15.1.5 文章生成工作流

第一步：素材收集

```python
# Agent自动执行
def collect_materials(topic):
    # 1. 搜索相关资讯（串行搜索，避免触发限制）
    news = search_news(topic, source=['google', 'social_media', 'reddit'])

    # 2. 从记忆库找相关经历
    memories = search_memory(keywords=['copilot', '逆向', '出海'])

    # 3. 拉取技术文档
    docs = fetch_tech_docs(topic)

    return {
        'news': news,
        'memories': memories,
        'docs': docs
    }
```

第二步：按风格写初稿

```
写作风格要求：

开头模式：
- 第一句："大家好，我是孟健。"
- 第二句：必须"炸"（数据、反差、冲突）

结构要求：
- 模块化分段，编号清晰
- 每500字一句可截图金句
```

```
- 所有场景用before vs after对比
- 2000-3000字控制篇幅

标题公式：
- 数字 + 成果 + 反差
- 示例："10分钟完成全平台发布，效率提升94%"

结尾要求：
- "弹射"而不是"降落"
- 一句狠话收束
- 示例："把重复的交给系统，把判断留给自己。"
```

第三步：自动推送飞书

```python
# Agent自动执行
def push_to_feishu(article):
    # 1. 调用飞书API创建文档
    doc = feishu.create_doc(
        title=article['title'],
        content=article['content']
    )

    # 2. Markdown转飞书格式
    formatted_content = convert_md_to_feishu(article['content'])

    # 3. 图片自动上传
    for img in article['images']:
        feishu.upload_image(doc.id, img)

    # 4. 生成链接发送
    link = feishu.get_doc_link(doc.id)
    telegram.send_message(f"文章已生成：{link}")

    return link
```

15.1.6 一键发布14个平台

使用字流：

```
1. 从飞书复制Markdown到字流编辑器
   ↓
```

```
2. 字流自动适配各平台格式:
   - 知乎: 图片居中、代码块优化
   - 掘金: 技术标签、代码高亮
   - B站: 视频封面、分P标题
   - 小红书: emoji优化、话题标签
   - 公众号: 排版美化、阅读原文
   - ... 共14个平台
   ↓
3. 点"一键发布"
   ↓
4. Chrome扩展自动填充各平台编辑器
   ↓
5. 10分钟全部发完
```

最新升级：API直接对接

```python
# Agent直接调用字流API
def publish_to_all_platforms(article):
    # 1. 推送到字流
    draft = ziliu_api.create_draft(
        title=article['title'],
        content=article['content'],
        images=article['images']
    )

    # 2. 自动发布
    result = ziliu_api.publish(
        draft_id=draft.id,
        platforms=['zhihu', 'juejin', 'bilibili', 'xiaohongshu', ...]
    )

    # 3. 返回发布结果
    return result
```

流程优化：

```
原流程：
Agent写完 → 推送飞书 → 打开飞书 → 复制内容 → 打开字流 → 粘贴 → 发布

优化后：
Agent写完 → 直接调用字流API → 自动发布

省去3个手动步骤！
```

15.1.7 关键细节

1. 风格文件是核心

```
# 写作风格规范.md

## 开头模式
- 第一句固定："大家好，我是孟健。"
- 第二句必须"炸"：
  ✅ "我现在写一篇文章，从选题到全平台发布，只要10分钟。"
  ✅ "Claude一个插件，让全球软件股蒸发2850亿美元。"
  ❌ "今天来讨论AI编程。"（太平淡）

## 标题公式
模式1：数字 + 成果 + 反差
- "10分钟完成全平台发布，效率提升94%"
- "从崩溃4671次到省50% token"

模式2：冲突 + 结果
- "Claude一个插件，让全球软件股蒸发2850亿美元"
- "Vibe Coding已死，Karpathy说未来叫Agentic Engineering"

模式3：疑问 + 答案
- "为什么我不再手动写文章？因为Agent比我先开始办公"

## 结构要求
- 模块化分段，用数字编号（01、02、03...）
- 每个模块500-800字
- 每500字必须有一句可截图金句
- 所有场景用before vs after对比

## 金句要求
- 短（10-20字）
- 狠（有冲击力）
```

```
- 可截图（视觉效果好）

示例：
✅ "把重复的交给系统，把判断留给自己。"
✅ "选题找我，不是我找选题。"
✅ "Agent比我先开始办公。"
❌ "我觉得AI很有用。"（太平淡）

## 结尾要求
- "弹射"而不是"降落"
- 一句狠话收束
- 留下思考或行动指引

示例：
✅ "如果你也在做个人IP，还在手动写、手动发、手动选题——想想看，你真正的价值是'敲键盘打字'，还是'判断什么值得说'？"
❌ "今天的分享就到这里，谢谢。"（太平淡）

## 用词习惯
- 多用短句，少用长句
- 多用动词，少用形容词
- 多用数据，少用感觉
- 多用对比，少用描述

## 禁用词汇
❌ "非常"、"很"、"特别"（太虚）
❌ "可能"、"也许"、"大概"（不确定）
❌ "我觉得"、"我认为"（太主观）
✅ 用数据代替感觉："效率提升94%"而不是"效率提升很多"
```

2. 记忆系统很重要

```
# memory/2026-02-11.md

## 今日工作
- 选题：Claude Opus 4.6发布
- 文章：《Claude一个插件，让全球软件股蒸发2850亿美元》
- 发布平台：14个
- 阅读数据：
  - 知乎：5,230阅读
  - 掘金：3,120阅读
  - 公众号：2,890阅读

## 使用数据
```

```
- 搜索次数：15次
- 生成字数：2,850字
- 修改次数：3次
- 耗时：8分钟

## 经验记录
- 标题中的"2850亿美元"数据很吸引眼球
- 开头的"炸弹"效果好，转发率高
- 结尾的"弹射"引发了很多讨论

## 下次改进
- 可以增加更多实战案例
- 技术细节可以再深入一些
```

3. 绝不自动发布

```
铁规则：所有内容必须经人工确认才能发布

原因：
- AI可能出现事实错误
- 措辞可能不当
- 判断可能有偏差

流程：
Agent写完 → 推送给我 → 我审核修改 → 确认后发布

AI负责效率，人负责质量底线。
```

4. cron定时任务是灵魂

```
# crontab -e

# 每天早上9点推送选题
0 9 * * * /usr/local/bin/openclaw run daily-topic-push

# 每天晚上11点生成工作日志
0 23 * * * /usr/local/bin/openclaw run daily-summary

# 每周一早上8点生成周报
0 8 * * 1 /usr/local/bin/openclaw run weekly-report
```

为什么定时任务重要？

```
没有定时任务：
- 想起来了才用
- 容易忘记
- 不成系统

有定时任务：
- 每天9点选题推过来
- 你不得不面对它
- 被推着走，效率高10倍
```

15.1.8 实际效果数据

全平台数据：

平台	粉丝数	月阅读量	月增长
公众号	15,000+	50,000+	+25%
知乎	8,000+	80,000+	+35%
掘金	5,000+	30,000+	+20%
B站	3,000+	20,000+	+40%
小红书	2,000+	15,000+	+50%

效率对比：

指标	使用前	使用后	提升
单篇耗时	3小时	10分钟	94.4%
每周产出	2-3篇	7篇	200%+
平台覆盖	3-4个	14个	300%+
内容质量	人工	AI+人工	持平

最大的变化不是速度，是心态：

```
以前：
- 写文章是个"重决策"
- 要不要写？写什么?什么时候写？
- 经常拖延、焦虑

现在：
- 写文章是个"轻决策"
- 选题已经在那了
- 日更就好，太轻松了！
```

15.1.9 可复制的配置模板

SOUL.md模板：

```
# Agent角色定义

## 身份
你是"墨笔"，一个专业的内容创作助手。

## 核心职责
1. 每天9点推送5个选题
2. 根据选定的选题生成初稿
3. 自动推送到飞书
4. 记录工作日志

## 写作风格
- 开头："大家好，我是[作者名]。"
- 第二句必须"炸"（数据、反差、冲突）
- 结构：模块化分段，编号清晰
- 每500字一句可截图金句
- 结尾："弹射"而不是"降落"

## 工具权限
- 可以调用：搜索API、飞书API、字流API
- 不可以：自动发布（必须人工确认）

## 工作流程
1. 搜索热点 → 生成选题 → 推送Telegram
2. 收到指令 → 收集素材 → 生成初稿
3. 推送飞书 → 等待确认 → 记录日志

## 边界
```

```
- 只管内容创作，不管其他
- 所有发布必须经人工确认
- 遇到不确定的事实，标注[待核实]
```

定时任务脚本：

```bash
#!/bin/bash
# daily-topic-push.sh

# 1. 搜索热点
openclaw ask "搜索过去24小时AI编程领域的热点，生成5个选题"

# 2. 推送到Telegram
openclaw telegram send "今日选题推荐：\n\n[选题内容]"

# 3. 记录日志
echo "$(date): 选题推送完成" >> /var/log/openclaw/daily-push.log
```

字流API集成：

```python
# ziliu_integration.py

import requests

class ZiliuAPI:
    def __init__(self, api_key):
        self.api_key = api_key
        self.base_url = "https://api.ziliu.example/v1"  # 示例API地址

    def create_draft(self, title, content, images=None):
        """创建草稿"""
        response = requests.post(
            f"{self.base_url}/drafts",
            headers={"Authorization": f"Bearer {self.api_key}"},
            json={
                "title": title,
                "content": content,
                "images": images or []
            }
        )
        return response.json()

    def publish(self, draft_id, platforms):
```

```python
        """发布到多个平台"""
        response = requests.post(
            f"{self.base_url}/publish",
            headers={"Authorization": f"Bearer {self.api_key}"},
            json={
                "draft_id": draft_id,
                "platforms": platforms
            }
        )
        return response.json()

# 使用示例
ziliu = ZiliuAPI("your-api-key")

# 创建草稿
draft = ziliu.create_draft(
    title="10分钟完成全平台发布",
    content="# 文章内容...",
    images=["image1.jpg", "image2.jpg"]
)

# 发布到所有平台
result = ziliu.publish(
    draft_id=draft['id'],
    platforms=['zhihu', 'juejin', 'bilibili', 'xiaohongshu']
)
```

15.2 案例2：AI 助手矩阵 - 多机器人多 Agent 模式

15.3.1 为什么需要多 Agent?

作为超级个体创业者，你可能需要不同类型的 AI 助手来处理不同的工作：

- **主助理**：使用最强大的模型（Claude Opus）处理复杂任务
- **内容创作助手**：专注于文章写作、文案创作
- **技术开发助手**：处理代码开发、技术问题
- **AI 资讯助手**：快速获取和整理 AI 行业动态

传统的单 Agent 模式需要频繁切换模型和上下文，效率低下。多 Agent 模式让你可以同时拥有多个专业助手，各司其职。

15.3.2 实现方案：多 Gateway + 多飞书机器人

核心思路：

- 创建 4 个飞书机器人应用
- 启动 4 个独立的 OpenClaw Gateway
- 每个 Gateway 连接一个飞书机器人
- 每个 Gateway 使用不同的 Agent 和模型

优势：

- ✅ 完全独立，互不干扰
- ✅ 直接私聊不同机器人即可切换 agent
- ✅ 不需要群组配置
- ✅ 不需要手动切换命令
- ✅ 配置清晰，易于管理
- ✅ 可以独立重启某个 Gateway

架构图：

```
┌──────────────────────────────────────────────────────────┐
│                        飞书 (Feishu)                      │
├──────────────────────────────────────────────────────────┤
│  机器人1: 主助理          机器人2: 内容创作助手          │
│  机器人3: 技术开发助手    机器人4: AI资讯助手            │
└──────────────────────────────────────────────────────────┘
                          ↓ WebSocket
┌──────────────────────────────────────────────────────────┐
│                    OpenClaw Gateway 层                    │
├──────────────┬──────────────┬──────────────┬────────────┤
│ Gateway 1    │ Gateway 2    │ Gateway 3    │ Gateway 4  │
│ 端口: 18789  │ 端口: 18790  │ 端口: 18791  │ 端口: 18792│
│ Profile:     │ Profile:     │ Profile:     │ Profile:   │
│ main-        │ content-     │ tech-dev     │ ai-news    │
│ assistant    │ creator      │              │            │
└──────────────┴──────────────┴──────────────┴────────────┘
                          ↓
```

```
┌──────────────────────────────────────────────────────────┐
│                        Agent 层                          │
├──────────────┬──────────────┬──────────────┬────────────┤
│ main-agent   │ content-agent│ tech-agent   │ainews-agent│
│ Claude Opus  │ Claude Sonnet│ Claude Sonnet│ Gemini 2.5 │
│ 4.6 Thinking │ 4.5          │ 4.5 Thinking │ Flash      │
└──────────────┴──────────────┴──────────────┴────────────┘
```

15.3.3 配置步骤

第一步：创建飞书机器人应用

在飞书开放平台创建 4 个机器人应用，获取各自的 App ID 和 App Secret。

第二步：配置 Agent

为每个 Agent 创建配置文件（USER.md 和 SOUL.md），定义角色和职责。

第三步：运行配置脚本

使用自动化脚本创建多 Gateway 配置（详细脚本见附录）。

第四步：启动所有 Gateway

```
# 启动所有 Gateway
./start-all-gateways.sh

# 检查状态
./check-gateways.sh
```

15.3.4 使用方法：直接私聊机器人

这是最简单的使用方式：

1. **处理复杂任务** - 在飞书中搜索"主助理"机器人，直接发送消息
2. **创作内容** - 搜索"内容创作助手"机器人，发送写作需求
3. **开发代码** - 搜索"技术开发助手"机器人，发送技术问题
4. **获取资讯** - 搜索"AI资讯助手"机器人，请求最新动态

关键优势：不需要任何手动切换，直接私聊对应的机器人即可！

15.3.5 实战案例：内容创作工作流

场景：写一篇技术文章

1. **构思阶段** - 私聊"主助理"：讨论文章主题和大纲（Claude Opus 深度思考）
2. **写作阶段** - 私聊"内容创作助手"：撰写文章内容（Claude Sonnet 快速生成）
3. **代码示例** - 私聊"技术开发助手"：编写代码示例（Claude Sonnet Thinking 确保质量）
4. **资讯补充** - 私聊"AI资讯助手"：获取最新技术动态（Gemini Flash 快速检索）

15.3.6 性能和成本

资源占用：

- 每个 Gateway 约 400MB 内存
- 4 个 Gateway 总共约 1.6GB
- 对于 64GB 内存的机器完全可以接受

成本分析（使用自建 API 代理）：

Agent	模型	用途	月使用量	月成本
main-agent	Claude Opus 4.6	复杂任务	100万 tokens	$15
content-agent	Claude Sonnet 4.5	内容创作	200万 tokens	$6
tech-agent	Claude Sonnet 4.5	技术开发	150万 tokens	$4.5
ainews-agent	Gemini 2.5 Flash	资讯获取	300万 tokens	$0
总计	-	-	750万 tokens	**$25.5**

15.3.7 核心价值

效率提升：

- 不需要频繁切换模型和上下文
- 每个 Agent 专注自己的领域
- 并行处理多个任务

成本优化：

- 简单任务使用 Gemini Flash（免费）
- 复杂任务才使用 Claude Opus
- 内容创作使用 Claude Sonnet（性价比高）

稳定可靠：

- 完全独立，互不干扰
- 可以独立重启某个 Gateway
- 配置清晰，易于管理

15.5 案例3：1天冷启动100人付费社群

15.3.1 案例背景

创业者：某创业者（AI编程API服务）
产品：某AI编程API服务平台
痛点：工具型产品，用户用完就走，没有粘性
目标：建立付费社群，形成增长飞轮
效果：2天从策划到100人，收入¥4,900+

15.3.2 增长飞轮模型

```
工具（API服务平台）
  ↓
内容（教程/分享）
  ↓
社群（交流群）
  ↓
更多人用工具
  ↓
（循环）
```

核心逻辑：

```
问题：
- 用户用完就走
- 没有粘性
- 难以复购

解决方案：
- 把用户聚到一起
- 互相帮忙踩坑
- 分享玩法
- 顺便用API服务

结果：
- 用户留存提升
- 口碑传播
- 自然复购
```

15.3.3 定价策略

最终定价：¥49/人

定价逻辑：

```
¥49 = 过滤器 + 钩子

过滤器：
- 不想做客服群
- 不想被"这个怎么配置"的问题淹没
- 愿意掏49块的人，至少是认真想玩的

```

```
钩子：
- 送$50 API额度
- 用户付¥49（约$7），拿到$50额度
- 数学上他赚了
- 心理上没有"花钱买了个群"的感觉

导流：
- $50额度会导流到API服务平台
- 用户注册、试用、习惯了之后自然续费
- 红包是获客成本
- 额度是钩子
- API消费是LTV
```

为什么不用阶梯定价？

```
考虑过的方案：
- 前50人免费
- 50-100人¥29
- 100人后¥49

放弃原因：
- 太复杂
- 免费进来的人参与度低
- 不如直接¥49，简单粗暴
```

15.3.4 1天搭建的基础设施

1. 上手教程（Notion文档）

```
# OpenClaw + AI Go Code 上手教程

## 5步从零跑通

### 第1步：获取API Key
1. 访问你的API服务平台
2. 注册账号
3. 进入控制台
4. 复制API Key

### 第2步：安装 OpenClaw
```bash
```
```

```
# macOS/Linux
curl -fsSL https://openclaw.example/install.sh | bash

# Windows
iwr https://openclaw.example/install.ps1 | iex
```

第3步：接入模型

```
# 配置API Key
openclaw config set api.key "your-api-key"

# 测试连接
openclaw ask "你好"
```

第4步：连接Telegram

```
# 创建Bot
openclaw telegram create-bot

# 绑定Bot
openclaw telegram bind
```

第5步：切换模型

```
# 查看可用模型
openclaw models list

# 切换到Claude
openclaw config set model "claude-opus-4"

# 切换到Codex
openclaw config set model "codex-5.3"
```

完成！

现在你可以在Telegram上和OpenClaw对话了。

```

**2. 群公告（2个版本）**

```markdown
# 群公告 v1（社交媒体版）

欢迎加入OpenClaw交流群！

这不是：
❌ 客服群
❌ 课程群
❌ 答疑群

这是：
✅ 玩家交流群
✅ 经验分享群
✅ 踩坑互助群

入群 = 认同以上

---

# 群公告 v2（微信版）

【OpenClaw交流群】

群规：
1. 没有老师，没有助教，没有答疑义务
2. 有问题先看教程，再问群友
3. 分享你的玩法，帮助他人成长
4. 禁止广告，禁止拉人，禁止灌水

额度领取：
1. 进群后发送：我的微信号
2. 等待管理员发放$50额度
3. 登录API服务平台查看

教程地址：
[Notion教程文档]
```
```

3. 自动化付款+进群

```python
# payment_automation.py

from flask import Flask, request
import qrcode
import requests

app = Flask(__name__)

@app.route('/pay', methods=['POST'])
def handle_payment():
    # 1. 接收付款通知
    payment_data = request.json

    # 2. 验证付款
    if verify_payment(payment_data):
        # 3. 生成群二维码
        qr_code = generate_group_qrcode()

        # 4. 发送给用户
        send_qrcode_to_user(
            user_id=payment_data['user_id'],
            qr_code=qr_code
        )

        # 5. 记录到数据库
        save_to_database(payment_data)

        return {"status": "success"}

    return {"status": "failed"}

def verify_payment(data):
    """验证付款"""
    # 调用微信支付API验证
    pass

def generate_group_qrcode():
    """生成群二维码"""
    qr = qrcode.QRCode()
    qr.add_data("https://t.me/your_group")  # 替换为你的群链接
    qr.make()
    return qr.make_image()
```

```python

def send_qrcode_to_user(user_id, qr_code):
    """发送二维码给用户"""
    # 通过微信/Telegram发送
    pass
```

关键点：降低用户操作成本

```
流程优化：
看到海报 → 扫码付款 → 自动弹出群二维码 → 进群

越短、越顺滑，转化率越高
```

4. 额度发放系统

```python
# credit_distribution.py

import pandas as pd
from datetime import datetime

class CreditManager:
    def __init__(self):
        self.db = pd.DataFrame(columns=[
            'user_id', 'wechat', 'amount', 'status', 'created_at'
        ])

    def add_user(self, user_id, wechat):
        """添加用户"""
        self.db = self.db.append({
            'user_id': user_id,
            'wechat': wechat,
            'amount': 50,
            'status': 'pending',
            'created_at': datetime.now()
        }, ignore_index=True)

    def distribute_credit(self, user_id):
        """发放额度"""
        # 1. 检查是否已发放
        if self.is_distributed(user_id):
            return {"error": "已发放"}

        # 2. 调用API发放
```

```python
        result = api_distribute_credit(user_id, amount=50)

        # 3. 更新状态
        self.db.loc[self.db['user_id'] == user_id, 'status'] = 'distributed'

        return result

    def is_distributed(self, user_id):
        """检查是否已发放"""
        row = self.db[self.db['user_id'] == user_id]
        return len(row) > 0 and row.iloc[0]['status'] == 'distributed'

    def export_report(self):
        """导出报告"""
        return self.db.to_csv('credit_report.csv')

# 使用示例
manager = CreditManager()

# 添加用户
manager.add_user('user123', 'wechat_abc')

# 发放额度
manager.distribute_credit('user123')

# 导出报告
manager.export_report()
```

为什么需要自动化?

```
100人的额度管理，靠人工迟早出错：
- 重复发放
- 漏发
- 记录混乱

自动化系统：
- 防止重复
- 自动记录
- 一键导出
```

15.3.5 推广策略

多平台同步发布：

```
社交媒体：
- 发了几条带海报的推文
- 主阵地，技术人群集中

公众号：
- 发了一篇图文
- 详细介绍群价值

没买量，没互推，纯自然流量
```

海报迭代：

```
第一版海报：
- 发出去没什么反应
- 转化率低

第二版海报（Claude Max调整）：
- 视觉更吸引
- 文案更清晰
- 加上微信支付自动化
- 转化立刻起来了

教训：
别低估"最后一公里"的体验
```

15.3.6 5个Telegram Bot矩阵

Bot分工：

```
1. 小O（私人助理）
   - 管配置
   - 管记忆
   - 管提醒

2. 内容Bot
   - 每天搜热点
```

```
    - 写推文草稿
    - 生成长文大纲
    - 自动写入Notion

3. 出海Bot
    - 专注AI编程出海方向
    - 深度研究
    - 竞品分析

4. 学习Bot
    - 教我怎么玩OpenClaw
    - 教群友使用技巧
    - 整理FAQ

5. 团队Bot
    - 管员工档案
    - 管KPI
    - 管薪酬
    - 管周报
```

为什么要多Bot?

```
单Bot问题:
- 上下文混乱
- 角色不清晰
- 容易出错

多Bot优势:
- 各管一摊
- 独立运行
- 独立上下文
- 互不干扰

相当于雇了5个AI员工，7×24在线
```

Bot配置示例:

```
# 内容Bot配置

## 角色
你是内容Bot，专门负责内容创作相关的工作。

## 职责
```

```
1. 每天9点搜索AI编程热点
2. 生成5个选题推送给我
3. 根据选定的选题生成推文草稿
4. 生成长文大纲
5. 自动写入Notion

## 工具权限
- 可以调用：搜索API、Notion API、社交媒体API
- 不可以：发布内容（必须人工确认）

## 工作流程
1. 定时搜索热点
2. 生成选题
3. 推送Telegram
4. 等待指令
5. 生成内容
6. 写入Notion

## 输出格式
选题格式：
【热度⭐⭐⭐⭐⭐】标题
角度：切入角度
要点：核心要点

推文格式：
- 第一句：吸引眼球
- 中间：核心内容
- 最后：行动指引
- 字数：280字以内
```

15.3.7 模型自动切换

看门狗脚本：

```python
# model_watchdog.py

import time
import requests
from datetime import datetime

class ModelWatchdog:
    def __init__(self):
```

```python
        self.models = {
            'claude-opus-4': {
                'url': 'https://api.example.com/v1/claude',  # 替换为你的API地址
                'backup': 'claude-sonnet-4'
            },
            'codex-5.3': {
                'url': 'https://api.example.com/v1/codex',  # 替换为你的API地址
                'backup': 'codex-5.2'
            }
        }
        self.current_model = 'claude-opus-4'

    def check_health(self, model):
        """检查模型健康状态"""
        try:
            response = requests.get(
                self.models[model]['url'] + '/health',
                timeout=5
            )
            return response.status_code == 200
        except:
            return False

    def switch_model(self, from_model, to_model):
        """切换模型"""
        # 1. 更新配置
        update_config('model', to_model)

        # 2. 通知Telegram
        send_telegram_message(
            f"⚠️ 模型切换\n"
            f"从: {from_model}\n"
            f"到: {to_model}\n"
            f"时间: {datetime.now()}"
        )

        # 3. 记录日志
        log(f"Model switched: {from_model} -> {to_model}")

    def run(self):
        """运行看门狗"""
        while True:
            # 检查当前模型
```

```
            if not self.check_health(self.current_model):
                # 切换到备用模型
                backup = self.models[self.current_model]['backup']
                self.switch_model(self.current_model, backup)
                self.current_model = backup

            # 检查备用模型是否恢复
            for model in self.models:
                if model != self.current_model:
                    if self.check_health(model):
                        # 切回主模型
                        self.switch_model(self.current_model, model)
                        self.current_model = model

            # 等待5分钟
            time.sleep(300)

# 启动看门狗
watchdog = ModelWatchdog()
watchdog.run()
```

效果:

```
半夜模型出问题:
- 自动检测
- 自动切换
- Telegram通知
- 早上醒来，已经处理好了

不需要人工干预!
```

15.3.8 Notion全自动工作流

自动化内容:

```
1. 每天的工作日志
   - 做了什么
   - 用了什么工具
   - 遇到什么问题
   - 解决方案

```

```
2. 内容草稿
   - 选题
   - 大纲
   - 草稿
   - 发布记录

3. 选题库
   - 热点追踪
   - 选题评分
   - 使用状态

4. 群成员额度管理
   - 用户ID
   - 微信号
   - 额度状态
   - 发放时间

5. 社交媒体KOL监控
   - KOL列表
   - MRR追踪
   - 用户数追踪
   - 产品迭代记录
```

自动化脚本：

```python
# notion_automation.py

from notion_client import Client

class NotionAutomation:
    def __init__(self, token):
        self.client = Client(auth=token)

    def create_daily_log(self, content):
        """创建每日日志"""
        self.client.pages.create(
            parent={"database_id": "daily-log-db-id"},
            properties={
                "Date": {"date": {"start": datetime.now().isoformat()}},
                "Title": {"title": [{"text": {"content": f"日志 {datetime.now().date()}"}}]}
            },
            children=[
```

```
                {
                    "object": "block",
                    "type": "paragraph",
                    "paragraph": {"rich_text": [{"text": {"content": content}}]}
                }
            ]
        )

    def add_topic(self, topic, score, status="pending"):
        """添加选题"""
        self.client.pages.create(
            parent={"database_id": "topics-db-id"},
            properties={
                "Topic": {"title": [{"text": {"content": topic}}]},
                "Score": {"number": score},
                "Status": {"select": {"name": status}}
            }
        )

    def update_credit_status(self, user_id, status):
        """更新额度状态"""
        # 查找用户记录
        results = self.client.databases.query(
            database_id="credits-db-id",
            filter={"property": "UserID", "rich_text": {"equals": user_id}}
        )

        if results['results']:
            page_id = results['results'][0]['id']
            # 更新状态
            self.client.pages.update(
                page_id=page_id,
                properties={"Status": {"select": {"name": status}}}
            )

# 使用示例
notion = NotionAutomation("your-notion-token")

# 创建日志
notion.create_daily_log("今天完成了群冷启动，100人入群")

# 添加选题
notion.add_topic("Claude Opus 4.6发布", score=5)
```

```

# 更新额度状态
notion.update_credit_status("user123", "distributed")
```

15.3.9 实际数据

冷启动数据：

指标	数据
策划到执行	2天
入群人数	100+
定价	¥49/人
红包收入	¥4,900+
额度成本	$5,000（自有平台，边际成本可控）
推广费用	¥0
退款	0

转化漏斗：

```
看到海报：约500人
  ↓ 20%
点击链接：约100人
  ↓ 100%
完成付款：100人
  ↓ 100%
进群：100人
```

关键指标：

```
付费转化率：20%（100/500）
退款率：0%
额度领取率：95%（95/100）
API激活率：60%（60/100）
```

15.3.10 踩坑与反思

坑1：海报和支付链路决定转化

```
第一版海报：
- 发出去没什么反应
- 转化率<5%

第二版海报（Claude Max调整）：
- 视觉更吸引
- 文案更清晰
- 加上微信支付自动化
- 转化率>20%

教训：
别低估"最后一公里"的体验
```

坑2：不要高估"免费"的价值

```
考虑过：
- 前50人免费进群当种子用户

放弃原因：
- 免费进来的人和付费进来的人
- 参与度完全不一样
- ¥49不多，但这个动作本身就是筛选
```

坑3：群公告要提前想好

```
第一版：
- 写得太长
- 没人看

第二版：
- 精简成几个要点
- 加上额度领取流程
- 效果好多了
```

15.5 一人公司的核心方法论

15.3.1 AI是真的颠覆生产力

案例1的感受：

```
有AI之前：
- 写一篇文章：3小时
- 每周产出：2-3篇
- 平台覆盖：3-4个
- 感觉：累、焦虑、拖延

有AI之后：
- 写一篇文章：10分钟
- 每周产出：7篇
- 平台覆盖：14个
- 感觉：轻松、高效、持续

最大的变化不是速度，是心态
```

案例2的感受：

```
有AI之前：
- 海报设计：找设计师，等3天
- 额度系统：找开发，等1周
- 文案优化：自己改，改半天
- 报名表单：用第三方工具，功能受限

有AI之后：
- 海报：Claude Max调，10分钟
- 额度系统：Claude Max写，30分钟
- 文案：Claude Max优化，5分钟
- 报名表单：Claude Max开发，20分钟

一天就跑通了整套系统
```

核心感悟：

```
自媒体要被颠覆了

不是内容本身被AI替代
而是内容生产的效率被AI拉高了一个数量级

以前需要一个团队干的活
现在一个人加上AI就能搞定
```

15.3.2 一人公司的3个核心能力

1. 判断力

```
AI负责执行，人负责判断

判断什么？
- 选题值不值得做
- 内容质量是否达标
- 策略方向是否正确
- 用户反馈如何响应

AI可以：
- 生成100个选题
- 写10篇文章
- 设计20个海报

但只有人能判断：
- 哪个选题最有价值
- 哪篇文章最符合定位
- 哪个海报最能转化
```

2. 系统化思维

```
不是"用AI做事"
而是"搭建AI系统"

区别：
用AI做事：
- 想起来了才用
- 每次都要重新指导
- 效率提升有限

```

```
搭建AI系统：
- 定时自动运行
- 有记忆、有风格
- 效率提升10倍+

案例1：
- 定时任务推送选题
- 风格文件保证质量
- 记忆系统避免重复

案例2：
- 5个Bot各管一摊
- 自动化支付进群
- 看门狗自动切换模型
```

3. 快速迭代能力

```
不要追求完美，先跑通最小闭环

案例1：
- 第一版风格文件：500字
- 第二版：1000字
- 第三版：3000字
- 持续优化，越来越好

案例2：
- 第一版海报：转化率5%
- 第二版海报：转化率20%
- 快速迭代，立刻见效

核心：
- 先干，再优化
- 数据驱动迭代
- 不要想太多
```

15.3.3 一人公司的4个关键系统

1. 内容生产系统

```
核心：
- 定时任务推送选题
```

```
- Agent生成初稿
- 人工审核修改
- 自动发布多平台

关键文件：
- SOUL.md：定义Agent角色
- 风格文件：保证内容质量
- 记忆系统：避免重复

效果：
- 3小时 → 10分钟
- 效率提升94%
```

2. 社群运营系统

```
核心：
- 自动化付款进群
- 自动化额度发放
- 多Bot矩阵管理
- Notion记录一切

关键组件：
- 支付系统：降低操作成本
- 额度系统：防止重复漏发
- Bot矩阵：各管一摊
- Notion：中央数据库

效果：
- 1天冷启动100人
- 0退款
```

3. 模型管理系统

```
核心：
- 多模型配置
- 自动健康检查
- 自动切换备用
- 实时通知

关键脚本：
- 看门狗脚本
- 模型配置文件
- 通知系统
```

```

效果：
- 半夜出问题自动处理
- 不需要人工干预
```

4. 数据记录系统

```
核心：
- 所有操作自动记录
- 自动写入Notion
- 定期生成报告
- 数据驱动决策

记录内容：
- 每日工作日志
- 内容发布记录
- 用户额度状态
- KOL监控数据

效果：
- 不需要手动记录
- 随时查看数据
- 数据驱动优化
```

15.3.4 一人公司的5个铁律

铁律1：绝不自动发布

```
原因：
- AI可能出现事实错误
- 措辞可能不当
- 判断可能有偏差

流程：
Agent生成 → 人工审核 → 确认后发布

AI负责效率，人负责质量底线
```

铁律2：定时任务是灵魂

```
为什么？
- 让系统"推着你走"
- 不依赖"想起来了才用"
- 形成稳定的工作节奏

案例：
- 每天9点推送选题
- 每天23点生成日志
- 每周一生成周报
```

铁律3：记录一切

```
为什么？
- 数据驱动决策
- 避免重复劳动
- 持续优化系统

记录什么？
- 每天做了什么
- 用了什么工具
- 遇到什么问题
- 效果如何

工具：
- Notion（结构化数据）
- 日志文件（原始数据）
- 截图（视觉记录）
```

铁律4：快速迭代

```
不要追求完美：
- 先跑通最小闭环
- 快速上线测试
- 根据反馈迭代

案例1：
- 第一版风格文件：500字
- 持续优化到3000字

案例2：
- 第一版海报：转化率5%
- 第二版海报：转化率20%
```

铁律5：专注核心价值

```
什么是核心价值？
- 判断（选题、质量、方向）
- 不是执行（写作、设计、发布）

把重复的交给系统
把判断留给自己

案例1：
- Agent负责：搜索、写作、发布
- 人负责：选题、审核、优化

案例2：
- AI负责：海报、系统、文案
- 人负责：定价、策略、运营
```

15.3.5 一人公司的成本结构

案例1成本分析：

成本项	月费用	说明
OpenClaw	¥0	开源免费
API费用	¥50-200	根据使用量
飞书	¥0	免费版够用
字流	¥99	月付
服务器	¥20	腾讯云轻量
总计	**¥169-319**	**月成本**

收益：

- 节省时间：每周20小时
- 时薪¥200计算：每月节省¥16,000
- ROI：50倍+

案例2成本分析：

成本项	费用	说明
OpenClaw	¥0	开源免费
API费用	¥100	5个Bot
Notion	¥0	免费版
额度成本	$5,000	自有平台，边际成本低
推广费用	¥0	自然流量
总计	**¥100 + $5,000**	**一次性成本**

收益：

- 红包收入：¥4,900
- 后续API消费：持续收入
- 用户LTV：¥200-500/人

- ROI：5-10倍

15.3.6 一人公司的增长飞轮

案例1的飞轮：

```
内容创作
  ↓
全平台发布
  ↓
粉丝增长
  ↓
影响力提升
  ↓
更多机会
  ↓
更多内容素材
  ↓
（循环）
```

案例2的飞轮：

```
工具产品
  ↓
内容教程
  ↓
付费社群
  ↓
用户使用工具
  ↓
口碑传播
  ↓
更多用户
  ↓
（循环）
```

核心要素：

```
1. 找到你的核心能力
   - 案例1：内容创作
   - 案例2：技术产品
```

```

2. 用AI放大这个能力
   - 案例1：10倍内容产出
   - 案例2：1天搭建系统

3. 建立增长飞轮
   - 案例1：内容→粉丝→影响力
   - 案例2：工具→内容→社群

4. 持续优化系统
   - 数据驱动
   - 快速迭代
   - 不断提升
```

15.5 可复制的自动化模板

15.4.1 内容创作自动化模板

1. 目录结构：

```
~/.openclaw/
├── workspace/
│   ├── SOUL.md              # Agent角色定义
│   ├── writing-style.md     # 写作风格规范
│   └── memory/              # 记忆系统
│       ├── 2026-02-11.md
│       ├── 2026-02-12.md
│       └── ...
├── scripts/
│   ├── daily-topic-push.sh  # 每日选题推送
│   ├── generate-article.sh  # 生成文章
│   └── publish-all.sh       # 发布到所有平台
└── config/
    ├── feishu.json          # 飞书配置
    ├── ziliu.json           # 字流配置
    └── platforms.json       # 平台配置
```

2. SOUL.md模板：

```
# Agent角色定义
```

```markdown

## 身份
你是"[Agent名称]"，一个专业的[职责]助手。

## 核心职责
1. [职责1]
2. [职责2]
3. [职责3]

## 工作风格
- [风格要求1]
- [风格要求2]
- [风格要求3]

## 工具权限
- 可以调用：[工具列表]
- 不可以：[限制列表]

## 工作流程
1. [步骤1]
2. [步骤2]
3. [步骤3]

## 边界
- [边界1]
- [边界2]
- [边界3]
```

3. 定时任务模板：

```
# crontab -e

# 每日选题推送（早上9点）
0 9 * * * /path/to/openclaw run daily-topic-push

# 每日工作日志（晚上11点）
0 23 * * * /path/to/openclaw run daily-summary

# 每周周报（周一早上8点）
0 8 * * 1 /path/to/openclaw run weekly-report

# 每月月报（每月1号早上9点）
0 9 1 * * /path/to/openclaw run monthly-report
```

4. 发布脚本模板：

```bash
#!/bin/bash
# publish-all.sh

# 1. 从飞书获取文章
article=$(openclaw feishu get-article "$1")

# 2. 推送到字流
draft_id=$(openclaw ziliu create-draft "$article")

# 3. 发布到所有平台
openclaw ziliu publish "$draft_id" \
  --platforms "zhihu,juejin,bilibili,xiaohongshu,wechat,csdn,segmentfault,jianshu,toutiao,baidu,sohu,163,sina,qq"

# 4. 记录发布日志
echo "$(date): Published article $1" >> /var/log/openclaw/publish.log
```

15.4.2 社群运营自动化模板

1. 目录结构：

```
~/.openclaw/
├── bots/
│   ├── personal-assistant/  # 私人助理Bot
│   ├── content-bot/         # 内容Bot
│   ├── learning-bot/        # 学习Bot
│   ├── team-bot/            # 团队Bot
│   └── outreach-bot/        # 出海Bot
├── automation/
│   ├── payment.py           # 支付自动化
│   ├── credit.py            # 额度管理
│   └── watchdog.py          # 模型看门狗
└── data/
    ├── users.csv            # 用户数据
    ├── credits.csv          # 额度记录
    └── logs/                # 日志文件
```

2. Bot配置模板：

```markdown
# Bot配置

## 角色
你是[Bot名称]，专门负责[职责]。

## 职责
1. [职责1]
2. [职责2]
3. [职责3]

## 工具权限
- 可以调用：[工具列表]
- 不可以：[限制列表]

## 工作流程
1. [步骤1]
2. [步骤2]
3. [步骤3]

## 输出格式
[格式说明]
```

3. 支付自动化模板：

```python
# payment_automation.py

from flask import Flask, request
import qrcode

app = Flask(__name__)

@app.route('/pay', methods=['POST'])
def handle_payment():
    # 1. 接收付款通知
    payment_data = request.json

    # 2. 验证付款
    if verify_payment(payment_data):
        # 3. 生成群二维码
        qr_code = generate_group_qrcode()

        # 4. 发送给用户
        send_qrcode_to_user(
```

```
            user_id=payment_data['user_id'],
            qr_code=qr_code
        )

        # 5. 记录到数据库
        save_to_database(payment_data)

        return {"status": "success"}

    return {"status": "failed"}

if __name__ == '__main__':
    app.run(port=5000)
```

4. 额度管理模板：

```
# credit_management.py

import pandas as pd
from datetime import datetime

class CreditManager:
    def __init__(self, db_path='data/credits.csv'):
        self.db_path = db_path
        self.load_database()

    def load_database(self):
        """加载数据库"""
        try:
            self.db = pd.read_csv(self.db_path)
        except:
            self.db = pd.DataFrame(columns=[
                'user_id', 'wechat', 'amount', 'status', 'created_at'
            ])

    def add_user(self, user_id, wechat, amount=50):
        """添加用户"""
        self.db = self.db.append({
            'user_id': user_id,
            'wechat': wechat,
            'amount': amount,
            'status': 'pending',
            'created_at': datetime.now()
        }, ignore_index=True)
```

```python
        self.save_database()

    def distribute_credit(self, user_id):
        """发放额度"""
        if self.is_distributed(user_id):
            return {"error": "已发放"}

        # 调用API发放
        result = api_distribute_credit(user_id, amount=50)

        # 更新状态
        self.db.loc[self.db['user_id'] == user_id, 'status'] = 'distributed'
        self.save_database()

        return result

    def is_distributed(self, user_id):
        """检查是否已发放"""
        row = self.db[self.db['user_id'] == user_id]
        return len(row) > 0 and row.iloc[0]['status'] == 'distributed'

    def save_database(self):
        """保存数据库"""
        self.db.to_csv(self.db_path, index=False)

    def export_report(self):
        """导出报告"""
        return self.db.to_csv('credit_report.csv')
```

15.4.3 快速开始指南

步骤1：选择场景

```
场景A：内容创作自动化
- 适合：自媒体、博主、内容创作者
- 核心：定时选题 + 自动生成 + 多平台发布
- 效果：3小时 → 10分钟

场景B：社群运营自动化
- 适合：创业者、产品经理、社群运营
- 核心：自动化付款 + 额度管理 + 多Bot矩阵
- 效果：1天冷启动100人

场景C：混合场景
- 结合A和B
- 内容+社群双轮驱动
```

步骤2：搭建基础设施

```bash
# 1. 安装 OpenClaw
curl -fsSL https://openclaw.example/install.sh | bash  # 替换为实际安装地址

# 2. 创建工作目录
mkdir -p ~/.openclaw/workspace
mkdir -p ~/.openclaw/scripts
mkdir -p ~/.openclaw/config

# 3. 复制模板文件
cp templates/SOUL.md ~/.openclaw/workspace/
cp templates/writing-style.md ~/.openclaw/workspace/
cp templates/*.sh ~/.openclaw/scripts/

# 4. 配置API
openclaw config set api.key "your-api-key"
openclaw config set model "claude-opus-4"
```

步骤3：配置定时任务

```
# 编辑crontab
crontab -e

# 添加定时任务
0 9 * * * /path/to/openclaw run daily-topic-push
0 23 * * * /path/to/openclaw run daily-summary
```

步骤4：测试运行

```
# 测试选题推送
openclaw run daily-topic-push

# 测试文章生成
openclaw ask "写一篇关于AI编程的文章"

# 测试发布
openclaw run publish-all "article-id"
```

步骤5：持续优化

```
1. 收集数据
   - 记录每次运行的结果
   - 分析效果数据

2. 优化配置
   - 调整风格文件
   - 优化提示词
   - 改进流程

3. 迭代升级
   - 根据反馈改进
   - 添加新功能
   - 提升自动化程度
```

📝 本章小结

通过2个真实的一人公司案例，学习了：

核心收获

案例1：内容创作自动化

- 效率提升：3小时 → 10分钟（94%）
- 关键系统：定时任务 + 风格文件 + 记忆系统
- 核心价值：把重复的交给系统，把判断留给自己

案例2：社群运营自动化

- 冷启动：1天100人，收入¥4,900+
- 关键系统：支付自动化 + 多Bot矩阵 + 模型看门狗
- 核心价值：用AI放大核心能力，建立增长飞轮

方法论总结

一人公司的3个核心能力：

1. 判断力（AI负责执行，人负责判断）
2. 系统化思维（搭建AI系统，而不是用AI做事）
3. 快速迭代能力（先跑通最小闭环，再持续优化）

一人公司的4个关键系统：

1. 内容生产系统
2. 社群运营系统
3. 模型管理系统
4. 数据记录系统

一人公司的5个铁律：

1. 绝不自动发布
2. 定时任务是灵魂
3. 记录一切
4. 快速迭代
5. 专注核心价值

实战启示

AI真的颠覆了生产力：

- 不是内容本身被AI替代
- 而是内容生产的效率被AI拉高了一个数量级
- 以前需要一个团队干的活，现在一个人加上AI就能搞定

一人公司的未来：

- 更多人会选择一人公司模式
- AI是最好的"员工"
- 核心竞争力是判断力和系统化思维

下一步行动

1. **选择场景**：内容创作 or 社群运营 or 混合
2. **搭建系统**：按照模板快速搭建
3. **测试运行**：跑通最小闭环
4. **持续优化**：数据驱动迭代

开始你的一人公司之旅，用AI放大你的核心能力！

全书完

恭喜你完成了《OpenClaw完全指南》的学习！

从基础入门到高级应用，从技术配置到商业实战，你已经掌握了OpenClaw的完整知识体系。

现在，是时候开始你自己的AI自动化之旅了！

💡 **记住**：

- AI负责效率，人负责判断
- 把重复的交给系统，把判断留给自己
- 快速行动，持续迭代

🚀 **祝你成功**！